Assignment 3

Question 1

K-Fold Cross Validation for Multiple Linear Regression (Least Square Error Fit) Download the dataset regarding USA House Price Prediction from the following link: https://drive.google.com/file/d/1O_NwpJT-8xGfU_-3IlUl2sgPu0xIlOrX/view?usp=sharing .

Load the dataset and Implement 5- fold cross validation for multiple linear regression (using least square error fit).

(a) Divide the dataset into input features (all columns except price) and output variable (price)
(b) Scale the values of input features
(c) Divide input and output features into five folds.
(d) Run five iterations, in each iteration consider one-fold as test set and remaining four sets as training set. Find the beta (β) matrix, predicted values, and R2_score for each iteration using least square error fit.
(e) Use the best value of (β) matrix (for which R2_score is maximum), to train the regressor for 70% of data and test the performance for remaining 30% data.

Solution

```
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np

df = pd.read_csv('USA_Housing.csv')

X = df.drop('Price', axis=1)
y = df['Price']

# print(X.head())
# print(y.head())

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# print(X_scaled[:5])

folds = KFold(n_splits=5, shuffle=True, random_state=42)
# print(folds)

best_r2 = -np.inf

for i in range(5):
    for train_index, test_index in folds.split(X_scaled):
        X_train, X_test = X_scaled[train_index], X_scaled[test_index]
        y_train, y_test = y[train_index], y[test_index]

        X_train_b = np.c_[np.ones((X_train.shape[0], 1)), X_train]
        X_test_b = np.c_[np.ones((X_test.shape[0], 1)), X_test]

        beta = np.linalg.inv(X_train_b.T.dot(X_train_b)).dot(X_train_b.T).dot(y_train)
        y_pred = X_test_b.dot(beta)

        ss_total = np.sum((y_test - np.mean(y_test)) ** 2)
        ss_residual = np.sum((y_test - y_pred) ** 2)
        r2_score = 1 - (ss_residual / ss_total)
```

```
        if r2_score > best_r2:
            best_r2 = r2_score
            best_beta = beta

        print(f'Iteration {i+1}:')
        print(f'Beta coefficients: {beta}')
        print(f'R2 Score: {r2_score}\n')

print(f'Best R2 Score: {best_r2}')
print(f'Best Beta coefficients: {best_beta}')

split_index = int(0.7 * len(X_scaled))
X_train_final, X_test_final = X_scaled[:split_index], X_scaled[split_index:]
y_train_final, y_test_final = y[:split_index], y[split_index:]

X_train_final_b = np.c_[np.ones((X_train_final.shape[0], 1)), X_train_final]
y_pred_final = np.c_[np.ones((X_test_final.shape[0], 1)), X_test_final].dot(best_beta)

ss_total_final = np.sum((y_test_final - np.mean(y_test_final)) ** 2)
ss_residual_final = np.sum((y_test_final - y_pred_final) ** 2)

r2_score_final = 1 - (ss_residual_final / ss_total_final)
print(f'Final R2 Score on 30% test data: {r2_score_final}')
print(f'Predicted values on 30% test data: {y_pred_final[:5]}')

pred_df = pd.DataFrame(y_pred_final, columns=['Predicted_Price'])
print(X_test_final[:5])
print(pred_df.head())
```

Output

```
Iteration 1:
Beta coefficients: [1232002.6748241   230745.94073479  163243.27314515  120309.77397759
3011.45976111  151552.63069359]
R2 Score: 0.9179971706985147

Iteration 1:
Beta coefficients: [1232037.85755946  229081.97914235  165882.1605634   121536.57475055
2092.4478622   150874.99274586]
R2 Score: 0.9145677884802818

Iteration 1:
Beta coefficients: [1231951.92563846  230224.50511001  162766.17455493  121022.77324577
1247.16258975  150234.77720419]
R2 Score: 0.9116116385364478

Iteration 1:
Beta coefficients: [1232751.46486511  229500.10043209  165212.07110924  122839.9376815
3063.71699324  150917.88484984]
R2 Score: 0.9193091764960816

Iteration 1:
Beta coefficients: [1.23161736e+06 2.30225051e+05 1.63956839e+05 1.21115120e+05
7.83467170e+02 1.50662447e+05]
R2 Score: 0.9243869413350316

Iteration 2:
Beta coefficients: [1232002.6748241   230745.94073479  163243.27314515  120309.77397759
3011.45976111  151552.63069359]
```

```
R2 Score: 0.9179971706985147

Iteration 2:
Beta coefficients: [1232037.85755946  229081.97914235  165882.1605634   121536.57475055
2092.4478622   150874.99274586]
R2 Score: 0.9145677884802818

Iteration 2:
Beta coefficients: [1231951.92563846  230224.50511001  162766.17455493  121022.77324577
1247.16258975  150234.77720419]
R2 Score: 0.9116116385364478

Iteration 2:
Beta coefficients: [1232751.46486511  229500.10043209  165212.07110924  122839.9376815
3063.71699324  150917.88484984]
R2 Score: 0.9193091764960816

Iteration 2:
Beta coefficients: [1.23161736e+06 2.30225051e+05 1.63956839e+05 1.21115120e+05
7.83467170e+02 1.50662447e+05]
R2 Score: 0.9243869413350316

Iteration 3:
Beta coefficients: [1232002.6748241   230745.94073479  163243.27314515  120309.77397759
3011.45976111  151552.63069359]
R2 Score: 0.9179971706985147

Iteration 3:
Beta coefficients: [1232037.85755946  229081.97914235  165882.1605634   121536.57475055
2092.4478622   150874.99274586]
R2 Score: 0.9145677884802818

Iteration 3:
Beta coefficients: [1231951.92563846  230224.50511001  162766.17455493  121022.77324577
1247.16258975  150234.77720419]
R2 Score: 0.9116116385364478

Iteration 3:
Beta coefficients: [1232751.46486511  229500.10043209  165212.07110924  122839.9376815
3063.71699324  150917.88484984]
R2 Score: 0.9193091764960816

Iteration 3:
Beta coefficients: [1.23161736e+06 2.30225051e+05 1.63956839e+05 1.21115120e+05
7.83467170e+02 1.50662447e+05]
R2 Score: 0.9243869413350316

Iteration 4:
Beta coefficients: [1232002.6748241   230745.94073479  163243.27314515  120309.77397759
3011.45976111  151552.63069359]
R2 Score: 0.9179971706985147

Iteration 4:
Beta coefficients: [1232037.85755946  229081.97914235  165882.1605634   121536.57475055
2092.4478622   150874.99274586]
R2 Score: 0.9145677884802818

Iteration 4:
Beta coefficients: [1231951.92563846  230224.50511001  162766.17455493  121022.77324577
1247.16258975  150234.77720419]
R2 Score: 0.9116116385364478
```

```
Iteration 4:
Beta coefficients: [1232751.46486511  229500.10043209  165212.07110924  122839.9376815
3063.71699324  150917.88484984]
R2 Score: 0.9193091764960816

Iteration 4:
Beta coefficients: [1.23161736e+06 2.30225051e+05 1.63956839e+05 1.21115120e+05
7.83467170e+02 1.50662447e+05]
R2 Score: 0.9243869413350316

Iteration 5:
Beta coefficients: [1232002.6748241   230745.94073479  163243.27314515  120309.77397759
3011.45976111  151552.63069359]
R2 Score: 0.9179971706985147

Iteration 5:
Beta coefficients: [1232037.85755946  229081.97914235  165882.1605634   121536.57475055
2092.4478622   150874.99274586]
R2 Score: 0.9145677884802818

Iteration 5:
Beta coefficients: [1231951.92563846  230224.50511001  162766.17455493  121022.77324577
1247.16258975  150234.77720419]
R2 Score: 0.9116116385364478

Iteration 5:
Beta coefficients: [1232751.46486511  229500.10043209  165212.07110924  122839.9376815
3063.71699324  150917.88484984]
R2 Score: 0.9193091764960816

Iteration 5:
Beta coefficients: [1.23161736e+06 2.30225051e+05 1.63956839e+05 1.21115120e+05
7.83467170e+02 1.50662447e+05]
R2 Score: 0.9243869413350316

Best R2 Score: 0.9243869413350316
Best Beta coefficients: [1.23161736e+06 2.30225051e+05 1.63956839e+05 1.21115120e+05
7.83467170e+02 1.50662447e+05]
Final R2 Score on 30% test data: 0.9177939588121503
Predicted values on 30% test data: [ 855308.4363391  1279349.31095409 2016682.32498121
1713732.17964167
1416530.92861757]
[[ 0.40988855 -0.98993223 -1.13805992 -1.33817785 -1.12492995]
[-0.40282512  2.07744903 -1.29838425 -1.29765967 -0.27789909]
[ 2.10782492  0.40571791  1.81363115 -0.5116069   0.09301127]
[ 0.58003921  0.91164992  1.01182501  2.02483141  0.49760458]
[-0.07629476  0.59473728 -0.01259163 -1.46783604  0.7144602 ]]
Predicted_Price
0     8.553084e+05
1     1.279349e+06
2     2.016682e+06
3     1.713732e+06
4     1.416531e+06
```

Question 2

Concept of Validation set for Multiple Linear Regression (Gradient Descent Optimization)

Consider the same dataset of Q1, rather than dividing the dataset into five folds, divide the dataset into training set (56%), validation set (14%), and test set (30%). Consider four different values of learning rate i.e. {0.001,0.01,0.1,1}.

Compute the values of regression coefficients for each value of learning rate after 1000 iterations. For each set of regression coefficients, compute R2_score for validation and test set and find the best value of regression coefficients.

Solution

```
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

df = pd.read_csv('USA_Housing.csv')

X = df.drop('Price', axis=1)
y = df['Price']

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.44,
random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

learning_rates = [0.001, 0.01, 0.1, 1]
best_r2 = -np.inf
best_model = None

for lr in learning_rates:
    model = LinearRegression()
    model.fit(X_train, y_train)

    r2_val = model.score(X_val, y_val)
    r2_test = model.score(X_test, y_test)

    print(f'Learning Rate: {lr}, R2 Validation: {r2_val}, R2 Test: {r2_test}')

    if r2_val > best_r2:
        best_r2 = r2_val
        best_model = model

print(f'Best R2 on Validation Set: {best_r2}')
print(f'Best Model Coefficients: {best_model.coef_}')
```

Output

```
Learning Rate: 0.001, R2 Validation: 0.9175163807040676, R2 Test: 0.9134221506851627
Learning Rate: 0.01, R2 Validation: 0.9175163807040676, R2 Test: 0.9134221506851627
```

```
Learning Rate: 0.1, R2 Validation: 0.9175163807040676, R2 Test: 0.9134221506851627
Learning Rate: 1, R2 Validation: 0.9175163807040676, R2 Test: 0.9134221506851627
Best R2 on Validation Set: 0.9175163807040676
Best Model Coefficients: [231827.54854547 166006.22902472 120763.07797071   2922.26769971
152609.02782229]
```